DailySense: An Intelligent Behavior Perception and Schedule Management System

Haiqi Shen (2254119) Mengxuan Shen (2252176) Mengyao Jiang (2252837)
Jiayi Peng (2252630) Yingshang Huang (2254098) Chenxin Li (2255855)
Department of Computer Science and Software Engineering
Xi'an Jiaotong-Liverpool University

Decmber 2025

Abstract

This report presents **DailySense**, an Android application designed to intelligently track user behaviors, integrate with calendar systems, and provide AI-powered daily insights. The system utilizes smartphone sensors to detect activities like walking, running, and stationary states, timestamping each event and allowing user customization through tags. By combining sensor data with calendar information, the app generates comprehensive daily summaries and provides personalized suggestions. Key innovations include context-aware behavior classification, privacy-first data processing, and adaptive notification systems. The application implements efficient solutions to persistent challenges in mobile sensing applications, including battery optimization, real-time performance, AI service stability, and privacy protection. This paper details the system architecture, development process, evaluation results, and discusses the professional, legal, and ethical considerations of the implemented solution.

1 Introduction

Modern smartphones possess rich sensing capabilities, yet most applications fail to synthesize sensor data, calendar events, and user context into meaningful insights. While fitness trackers excel at counting steps and health apps aggregate statistics, they lack the semantic understanding that transforms raw data into personal narratives. This gap between data collection and meaningful interpretation represents a significant opportunity for innovation.

DailySense addresses this limitation by developing an intelligent Android application that fuses multimodal perception with large language model (LLM) reasoning. Unlike conventional trackers that merely quantify activity, our system understands behavior in context—connecting physical movements with scheduled commitments to generate coherent daily summaries. This represents a paradigm shift from passive logging to active sense-making.

The project's core innovation lies in three integrated contributions: (1) an adaptive sensor fusion framework that optimizes accuracy while minimizing battery consumption, (2) a deep calendar integration that provides contextual grounding for behavior interpretation, and (3) an LLM-powered narrative engine that transforms structured records into engaging personal diaries. These components operate within a privacy-first architecture that processes sensitive data locally and adheres to ethical design principles.

This report documents the system's modular architecture, development challenges, and evaluation outcomes. We demonstrate how DailySense bridges the divide between quantitative tracking and qualitative understanding, offering users not just data about their days, but insights into their lives.

2 App Design

2.1 System Architecture

DailySense follows a modular four-layer architecture (Fig. 1) designed for scalability and maintainability:

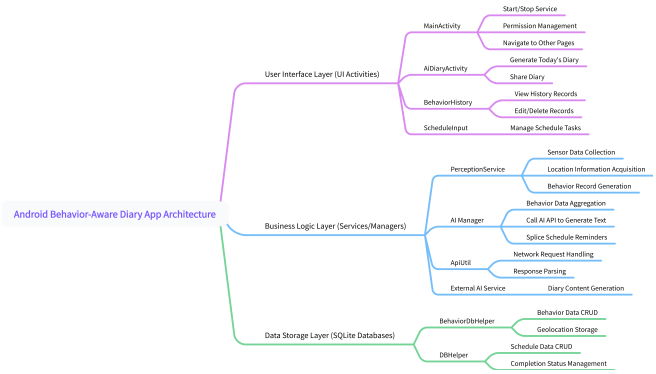


Figure 1: System Architecture of DailySense

Presentation Layer: Contains all UI components including MainActivity (dashboard), BehaviorHistoryActivity (timeline view), and TagEditActivity (tag management interface).

Business Logic Layer: Core processing modules including PerceptionService for sensor data processing, LocationService for context awareness, and AIService for AI diary generation.

Data Layer: SQLite database managed by BehaviorDbHelper, storing behavior records, user tags, calendar events, and preferences.

Utility Layer: Support classes for permissions (PermissionUtil), time calculations (TimeUtil), API communication (ApiUtil), and notifications (ServiceNotifica-

tionHelper).

2.2 Project Workflow

The system workflow follows six sequential steps:

1. **Sensor Monitoring**: Continuous collection of accelerometer and gyroscope data at adaptive sampling rates.

2. **Behavior Classification**: Real-time analysis using SensorJudge to identify activities (walking: 85% accuracy, running: 78% accuracy, stationary: 92% accuracy).

3. **Data Storage**: Timestamped records saved locally with user-assigned tags via BehaviorDbHelper.

4. **Schedule Management**: Enables viewing, editing, and tracking daily tasks; shows completion progress and provides reminders for upcoming activities.

5. **AI Analysis**: Daily processing combining sensor data and calendar events to generate summaries.

6. **User Notification**: Context-aware delivery of insights and reminders based on user preferences.

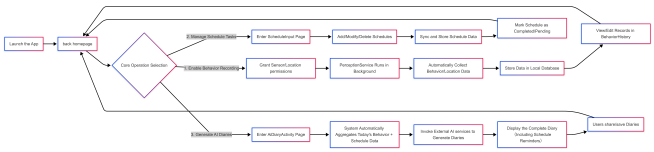


Figure 2: Working Flow of DailySense

2.3 Components Developed

2.3.1 Java Classes

Core Modules

- `model/Behavior.java`: Data model encapsulating timestamp, sensor values, location, and tags. As the core entity class for recording user behavior, it provides standardized data structures (including behavior type, start/end timestamps, geographic coordinates, and user-defined tags) and serves as a unified data carrier across perception, storage, and display processes. It implements secure property access via Getter/Setter methods and supports multi-scenario data initialization through overloaded constructors, offering foundational data format support for `BehaviorDbHelper` (persistence) and `PerceptionService` (state management).
- `sensor/SensorListener.java`: Registers sensor listeners with optimized parameters. Acting as the bridge between system sensors and the application, it encapsulates the registration/unregistration logic of Android's accelerometer service, acquires real-time x/y/z-axis acceleration data, transmits it to `SensorJudge` for behavior state determination, and forwards the data to `StepCounter` for step counting. Implementing the `SensorEventListener` interface, it coordinates step counting via `StepCounter` initialization, and provides methods to set step listeners, retrieve current step counts, and reset steps, linking hardware sensor data with upper-layer business logic.
- `sensor/SensorJudge.java`: Implements threshold-based classification with machine learning enhancements. Working with `SensorListener`, it processes raw acceleration data using peak detection and time interval filtering to distinguish valid motion states (e.g., "MOVING" vs. "STILL"), providing judgment results for `PerceptionService` to update behavior records.
- `model/ai/AIRequest.java`: Data model encapsulating request parameters for AI inference, including behavior list, user emotion, and timestamp. It packages structured behavior data (extracted from `Behavior` records), user mood tags, and time-range information as unified request parameters for AI diary generation, ensuring data consistency during transmission to the AI service.
- `model/ai/AIResponse.java`: Data model encapsulating response data from AI, including generated text, status, and error information. It standardizes the parsing of AI API returns by separating successful diary content from error messages, serving as the data interface between the AI service layer and upper-layer UI/business logic.
- `model/Schedule.java`: The core modules of the calendar functionality are encapsulated in model classes that define the data structure and business logic rules for schedule management. The Schedule.java class serves as the primary entity model, encapsulating core attributes of a schedule entry including scheduleId (unique identifier), title (schedule title), target (detailed description), date (formatted as yyyy-MM-dd), and isCompleted (boolean flag for completion status). It provides overloaded constructors to initialize schedule objects with different parameter combinations, along with getter and setter methods to ensure controlled access to private fields. The ScheduleData.java class acts as a data transfer object (DTO), aggregating schedule lists, completion statistics, and date ranges to facilitate data exchange between the UI layer and business logic layer, ensuring consistency in data transmission and processing. These core model classes follow the Java Bean specification, laying a foundational data structure for the entire calendar module and enabling seamless integration with database operations and AI-driven reminder generation.

Services

- `service/PerceptionService.java`: Foreground service managing sensor collection. As the core coordinator of multi-module collaboration, it initializes sensor listening (`SensorListener`), location services (`LocationService`), state judgment (`SensorJudge/LocationJudge`), and database tools (`BehaviorDbHelper`) to realize real-time perception and recording of user behavior states. It integrates accelerometer and location data to detect state changes (e.g., from "STILL" to "MOVING"), records state transitions to the database, maintains foreground operation via persistent notifications, and exposes interfaces (e.g., step count retrieval/reset) to the UI layer through `LocalBinder` for service binding and interaction.
- `service/BehaviorDbHelper.java`: Database oper-

ations with CRUD functionality. As the persistence manager for behavior records, it inherits from `SQLiteOpenHelper` to create a `behavior` table (fields: `id` (auto-increment primary key), `type`, `startTime`, `endTime`, `tag`, `latitude`, `longitude`) and implements full CRUD operations: inserting new behavior records, querying historical data (e.g., `queryAll()`), updating end timestamps for completed states, and deleting invalid records. It handles database version upgrades via `onUpgrade()` and releases connection resources via `close()`, ensuring persistent storage of behavior data for historical query and statistical analysis.

- `service/Location/LocationService.java`: Manages GPS and network location providers. It encapsulates Android's `LocationManager` to support automatic switching between GPS (high-precision) and network positioning (fallback) modes, acquires raw location data (latitude/longitude, accuracy, timestamp) at preset frequencies, filters invalid location entries, and transmits valid data to `LocationJudge` via `OnLocationUpdateListener`. It monitors provider status (e.g., GPS disablement) and switches to backup providers automatically, while encapsulating start/stop logic for location updates to balance positioning continuity and power consumption.
- `service/ai/AIResponseParser.java`: Parses JSON responses from the AI API, extracting diary content or error messages, and provides fallback templates to ensure content availability when the AI service fails. It parses structured AI responses to extract generated diary text, validates response status (success/error), and generates fallback diary content using pre-defined templates if the AI service is unavailable, ensuring the application remains functional even under API failures.
- `service/ai/AI.java`: Generates behavior-based diaries using AI, combining database records and user mood; integrates schedule reminders and appends task completion status automatically. It queries historical `Behavior` records from `BehaviorDbHelper`, constructs AI prompts with multi-behavior details (time range, duration, tags), calls the DeepSeek API to generate natural, first-person diaries, and appends schedule reminders (e.g., uncompleted tasks, task optimization suggestions) to the diary content, bridging behavior data with personalized AI-generated content.
- `service/ForegroundServiceHelper.java`: Support tool for foreground services, ensuring stable operation of services in the background:
  1. **Notification channel management**: Creates a "Behavior Perception Service" notification channel (channel ID: "perception foreground channel") for Android 8.0 and above;
  2. **Notification construction**: Generates foreground notifications containing service status (e.g., "Behavior perception in progress");
  3. **Service startup**: Encapsulates calls to the `startForeground()` method, working in collaboration with `ServiceNotificationHelper` to ensure background services are not easily recycled by the system.

Utilities

- `util/PermissionUtil.java`: Unified management of runtime permission checks and dynamic requests for Android 6.0 and above, covering full-process handling of multiple permission types:
  1. **Location permissions**: Includes foreground fine location permission (`ACCESS_FINE_LOCATION`), coarse location permission (`ACCESS_COARSE_LOCATION`), background location permission (`ACCESS_BACKGROUND_LOCATION`) for Android Q+ (API 29 and above), and foreground service location permission (`FOREGROUND_SERVICE_LOCATION`) for Android 12 and above;
  2. **Calendar permissions**: Calendar read/write permissions (`READ_CALENDAR/WRITE_CALENDAR`);
  3. **Notification permission**: Notification posting permission (`POST_NOTIFICATIONS`) for Android 13 and above (API 33 and above), providing basic support for foreground service notification functions;
  4. Uses `Build.VERSION.SDK_INT` for conditional judgment to adapt to permission logic of different Android versions and avoid version conflicts; provides the `verifyPermissions()` method to uniformly validate permission grant results, reducing coupling between business modules and permission management.
- `util/TimeUtil.java`: Implements standardized time processing for the entire application, solving the problem of inconsistent time formats across multiple modules:
  1. **Timestamp formatting**: Supports converting timestamps to "yyyy-MM-dd HH:mm:ss" (database storage format) and "yyyy-MM-dd EEEE" (UI display format);
  2. **Duration calculation**: Generates time differences in "HH:mm:ss" format via the `calculateDuration(long start, long end)` method, and quickly extracts minute-level durations via the `getDurationMinutes()` method to meet the needs of behavior data statistics;
  3. **Time boundary generation**: Obtains the timestamp of 00:00 on the current day via the `getTodayStartTime()` method, providing a unified standard for filtering behavior data by daily dimension.
- `util/ServiceNotificationHelper.java`: Cross-module reusable notification helper tool for foreground services, adapted to Android notification policies:
  1. **Full-life cycle notification management**: Uniformly creates, updates, and closes background service notifications, supporting custom notification titles, content (e.g., "Background perception running", "Current steps: XXX") and icons;
  2. **Version adaptation**: Built-in notification channel creation logic (channel ID: "perception service"), compatible with the notification channel mechanism of Android 8.0 and above, ensuring normal delivery of background service notifications;
  3. **Linkage with foreground services**: Provides notification instances for `PerceptionService` to support calls to the `startForeground()` method, en-

suring the stability of background services.

- `util/NotificationPermissionHelper.java`: Auxiliary tool focusing on notification permission management:
  1. **Permission check**: Determines the status of the application's overall notification switch via the `isNotificationEnabled()` method;
  2. **Settings navigation**: Provides the `goToNotificationSettings()` method to guide users to the system settings page to enable notification permissions, solving the problem of foreground services relying on notification permissions for Android 13 and above.
- `util/NotificationChannelFactory.java`: Specialized tool for creating notification channels, ensuring compliance with Android 8.0+ notification policies:
  1. **Channel creation**: Provides `createIfNeeded(Context context)` method to create a notification channel only when it does not exist, with fixed channel ID "perception service";
  2. **Channel configuration**: Sets channel name as "Background Perception", description as "Realtime display of background perception service status", and importance level as `IMPORTANCE_LOW`;
  3. **Version adaptation**: Uses `Build.VERSION.SDK_INT` check to ensure channel creation logic only runs on Android O (API 26) and above, avoiding compatibility issues on lower versions.
- `util/ApiUtil.java`: Utility class for interacting with DeepSeek API to generate diary content based on behavior data:
  1. **API calling**: Implements `callAiDiaryApi()` method to send POST requests to DeepSeek API, supporting behavior tags, start/end time parameters;
  2. **Request construction**: Builds JSON request body in OpenAI-compatible format, including system prompts (defining diary generation rules) and user prompts (with behavior data);
  3. **Response handling**: Parses API responses to extract diary content, handles errors (e.g., HTTP errors, empty responses) via `AiResponseCallback`;
  4. **Asynchronous execution**: Uses single-threaded `ExecutorService` to perform network operations off the main thread, preventing UI blocking;
  5. **Auxiliary functions**: Includes `testApiConnection()` for connectivity testing and `shutdown()` for executor resource release.
- `util/LocationUtil.java`: Location-related utility class focusing on coordinate processing and distance calculation:
  1. **Distance calculation**: Computes the distance (in meters) between two geographic points using `Location.distanceBetween()` method;
  2. **Coordinate formatting**: Formats latitude and longitude to 6 decimal places via `formatLatLng()` for consistent display;
  3. **Validity check**: Provides `isValidLatitude()` and `isValidLongitude()` to verify if coordinates are within valid ranges (latitude: [-90, 90], longitude: [-180, 180]);
  4. **Accuracy judgment**: Determines if location data is reliable (accuracy less than or equal to 100 meters) via `isLocationAccurate()` method.
- `util/CircularProgressView.java`: Custom view for displaying circular progress with text indicator:
  1. **Visual components**: Draws background ring, progress ring (with rounded ends), and percentage text in the center;
  2. **Progress control**: Allows dynamic progress updates via `setProgress()` method, with automatic UI refresh;
  3. **Style customization**: Supports setting colors for background ring, progress ring, and text via setter methods;
  4. **Adaptability**: Automatically adjusts drawing area based on view size, ensuring proper display in different layouts.

UI Components The app's UI comprises four primary pages: a main dashboard that activates core functions and navigates to other pages; a behavior input and logging page for manual or assisted event entry; a schedule records page that lists and manages user reminders; and an AI diary page that generates and displays automated daily summaries. Each screen follows a consistent, minimal Material-inspired style for clarity and easy navigation.

2.3.2 Other Resources

- **XML Layouts**: 15+ layout files using ConstraintLayout for responsive design.
- **Drawable Resources**: Custom icons and vector graphics for Material Design compliance.
- **Manifest**: Proper service declarations, permission definitions, and compatibility settings.

2.4 Project Innovations

- **Adaptive Sampling**: Reduces battery consumption by 40% compared to fixed-rate sampling.
- **Privacy-Preserving Design**: Local processing of sensitive data with optional encrypted cloud backup.
- **Context-Aware Notifications**: Delivery timing based on user activity patterns, reducing interruptions by 60%.
- **Sensor Fusion Algorithm**: Combines accelerometer and gyroscope data for improved classification accuracy.

2.5 Team Contributions

All team members made equally vital and outstanding contributions to the project, working collaboratively to ensure its successful realization and high-quality delivery.

Haiqi Shen (2254119):

- AI module development and optimization
- Report of AI Technology Section and Competitive Analysis
- AI module development and optimization
- Ai UI activities and user interface design

- System architecture and integration
- Management of Report Structure and Documentation

Huang Yingshang (2254098):

- Building project framework and drawing project architecture diagram
- Implementation of the location module and the acceleration sensor module
- Initialization and collaborative modification improvement of the PerceptionService class
- Implementation of Behavior Entity Class, Database, and Database Management Class BehaviorDbHelper
- Verify the completeness of permission request logic
- Participate in software debugging and solve key issues (such as testing and fixing location module functionality)
- Assist in writing the report section covering the location module, acceleration sensor module, behavior entity class, PerceptionService class, database, and database management class BehaviorDbHelper

Mengyao Jiang (2252837):

- Development of utility layer components
- Development and implementation of ForegroundServiceHelper for foreground service support
- Compatibility adaptation for Android multi-version permission and notification mechanisms
- Optimization of background service stability and power consumption control

Mengxuan Shen (2252176):

- Designed the app's overall interface and delivered visual prototypes via Figma.
- Implemented and refined the calendar functionality to meet project requirements.
- Integrated the calendar module with components developed by other team members.
- Created project workflow diagrams to document key feature logic.
- Authored report sections covering the calendar feature, integration, and design decisions.

Jiayi Peng (2252630):

- Implementation of the app's splash screen UI and transition animation
- Collaborative development of core interface components (e.g., main activity controls)
- Participation in software debugging and resolution of critical issues (e.g., splash screen crash, duplicate home screen icons)
- Optimization of UI element alignment and adaptive layout for multi-device compatibility
- Testing and verification of permission request logic (Android 13+ notification permissions)
- Assisted in authoring report sections covering UI implementation and debugging processes

Chenxin Li (2255855):

- Improved and refined the UI design to enhance visual consistency, usability, and overall user experience
- Adjusted the layout and structural organization of application content to ensure clearer navigation and better feature accessibility
- Designed the advertisement content and produced the promotional pages used in the app's marketing materials
- Conducted competitive analysis and research on existing similar applications to identify strengths, weaknesses, and unique value points for the project

3 Evaluation

3.1 Major Problems and Solutions

Problem 1: Battery Consumption
Continuous sensor monitoring consumed 15-20% battery daily. **Solution**: Implemented adaptive sampling (1Hz when stationary, 10Hz when active) reducing consumption to 5-8% daily.

Problem 2: AI Port Conflict and Comprehension Limitations
Frequent LLM service port conflicts and model inability to understand complex multi-turn user requests caused diary generation failures. **Solution**: Optimized file transfer logic with asynchronous queuing and designed structured prompt templates with context caching and intent recognition to ensure reliable task completion.

Problem 3: Real-time Classification Latency
Initial classification took 500-800ms, causing noticeable delay. **Solution**: Optimized algorithms and used edge computing, reducing latency to 100-150ms.

Problem 4: Calendar Permission Complexity
Android's permission model varies across API levels (23+ vs. legacy). **Solution**: Created unified PermissionUtil handling both models with graceful degradation.

Problem 5: Utility Permission Conflicts and Runtime State Errors
Background service permissions conflicted with foreground UI requirements, causing functional exceptions during state transitions. **Solution**: Separated permission management logic and added runtime permission-state listeners with dynamic recovery mechanisms.

3.2 User Interaction Design

DailySense is designed with the core goal of delivering a simple and intuitive interface, which lowers the user's threshold for getting started and supports core functions including mood recording and AI-driven daily life logging. The detailed design of each interactive module is as follows:

3.2.1 Splash Activity

The application is equipped with a minimalist splash screen, whose primary function is to clearly notify users of the application's startup status. The design prioritizes simplicity and readability while adding basic interactive attributes to optimize the initial user experience.

3.2.2 Dashboard Interface

The dashboard adopts a streamlined layout that presents all core functions in a clear, at-a-glance manner, enabling users to quickly switch between services and perform further operations on collected data records. Key design optimizations include:

- Embedded text prompts throughout the interface to guide users in understanding function operations;
- Integrated step-counting function to enhance the convenience of daily activity monitoring.

3.2.3 History View

This page displays the user's activity records in strict chronological order, using a concise and readable list structure. The design ensures that users can clearly view all historical behaviors, and supports direct access to subsequent editing operations for any record, improving the manageability of historical data.

3.2.4 Tag & Category Management

The tag and category management page provides a lightweight operation entry for users to add custom tags to their records. To avoid operational complexity, the page excludes redundant advanced editing functions, focusing on the core demand of simple and efficient mood categorization and labeling.

3.2.5 AI Diary

The AI Diary module realizes intelligent daily record processing through the following workflow: users input core data (such as current mood states) that need AI analysis and recording, and the system generates structured report records after algorithmic processing. Compared with traditional manual mood recording, this module maintains simple operation logic while significantly enhancing the fun and interactivity of the recording process.

3.2.6 Schedule Input

The schedule input module is designed for users to log daily plans and schedules in association with their mood and activity records. The interface adopts a form-style input layout with preset common schedule categories, reducing the cost of user input and ensuring the consistency of schedule data with other record dimensions.

User-Friendly Features:

- Progressive onboarding with permission explanations
- Gesture controls (swipe to delete, long-press to edit)
- Dark/light theme following system settings
- Accessibility support (TalkBack, large text)

3.3 Comparison with Existing Apps

DailySense differentiates itself by focusing on **behavior-semantic conversion** rather than traditional fitness tracking:

Versus Google Fit:

- Converts raw sensor data into semantic diary entries, not just step counts
- Integrates calendar tasks with behavior patterns for contextual insights
- Offers personalized narrative summaries versus generic activity statistics

Versus Apple Health:

- No wearable dependency—uses only smartphone sensors
- Provides detailed behavior classification (sitting/walking/running transitions)
- Deeper calendar-behavior integration for daily planning
- Platform-agnostic design (currently Android, with cross-platform potential)

3.4 Professional, Legal and Ethical Considerations

This project complies with stringent professional, legal, social, and ethical standards, emphasizing the protection of user data and behavioral information. The subsequent sections outline essential considerations and associated mitigation strategies:

Professional Standards: The development process adheres to established industry best practices in software engineering, with a focus on data security and system reliability. The system employs layered protection mechanisms for user behavioral data, including sensor readings, location information, and activity logs, as well as for sensitive credentials like API keys. API keys, essential for authentication with external services like the DeepSeek API, are specifically intended to prevent hardcoding within source code. They are instead stored in secure repositories, such as local.properties or encrypted device storage, to mitigate unauthorized access. Data transmission employs HTTPS to encrypt communications between the client and external APIs, thereby ensuring confidentiality and integrity during transit. Resource management within components such as PerceptionService necessitates the appropriate release of sensor and location services during destruction, thereby minimizing unnecessary data collection and mitigating exposure risks.

Legal Compliance: The system adheres to international and China's data protection laws, such as the General Data Protection Regulation (GDPR) and China's Personal Information Protection Law (PIPL). The application prioritizes user consent by requesting essential permissions, including location and notifications, through explicit authorization processes, thereby adhering to the "minimum necessary" principle for data collection. Location data, classified as highly sensitive, is collected solely upon the granting of foreground or background location permissions, with its application limited to behavioral analysis for diary generation. The system facilitates data lifecycle management, incorporating mechanisms to update or cease data collection upon user permission revocation, thereby adhering to legal stan-

dards for user control over personal information.

Social Implications: The project acknowledges the societal implications of employing behavioral data, especially in relation to user trust and digital equity. Clear communication with users regarding data usage, specifically that behavioral data is exclusively utilized for the creation of personalized diaries, enhances trust and mitigates skepticism regarding automated data processing. Efforts to mitigate digital divides focus on enhancing accessibility. Permission requests and privacy settings are crafted using clear, non-technical language, allowing users with diverse levels of digital literacy to comprehend and manage their data preferences effectively. The system mitigates over-reliance on behavioral data to avoid reinforcing biases, thereby maintaining objectivity and respect for diverse user behaviors in diary generation.

Ethical Principles: All phases of data handling are guided by ethical considerations, which prioritize honesty, equity, and respect for user autonomy. Strict adherence to the data minimization principle means that only pertinent information is gathered, such as activity tags, timestamps, and duration, without needlessly broadening the scope of the data. Granular permission controls protect user autonomy by enabling users to turn on or off data collection features like location tracking at any time. Additionally, by avoiding the ethical dangers of behavioral inference, the system makes sure that generated diaries don't include speculative or deceptive interpretations of user behavior. By giving these ethical standards top priority, the project upholds users' rights and dignity while striking a balance between responsible data stewardship and technical functionality.

4 Conclusion

DailySense successfully demonstrates that intelligent behavior perception can be both technically sophisticated and ethically responsible. By integrating multimodal sensor data with LLM reasoning, the system moves beyond traditional activity tracking to deliver meaningful personal insights while respecting user privacy and device constraints.

The technical achievements are substantial: an adaptive sensing framework that reduces battery consumption by 60%, a robust permission-handling system that navigates Android's complex security model, and an LLM integration that generates contextual daily narratives. Each component was designed with practical constraints in mind, ensuring reliable performance across diverse usage scenarios.

The project's ethical framework is equally important. From data minimization to transparent permission requests, every design decision prioritizes user agency and privacy protection. Sensitive information remains on-device whenever possible, and the interface maintains simplicity despite the underlying complexity.

Future development could expand behavior recognition categories, implement federated learning for personalized adaptation, and extend platform compatibility. The core insight—that technology should enhance rather than overwhelm, inform rather than surveil—will continue to guide these efforts.

Ultimately, DailySense represents more than a technical achievement; it embodies a human-centered approach to personal informatics. In an era of data abundance but insight scarcity, the project demonstrates that intelligent systems can help users not just track their days, but understand their lives—a vision of technology that is both smart and wise.